

# SDN QoS Project - RSX217 CNAM Paris

---

## Contrôleur SDN et Gestion de la Qualité de Service (QoS)

**Auteur:** GAVI Holali David

**Cours:** RSX217 - Mini-projet #19

**Date:** Décembre 2025

---

## Description du Projet

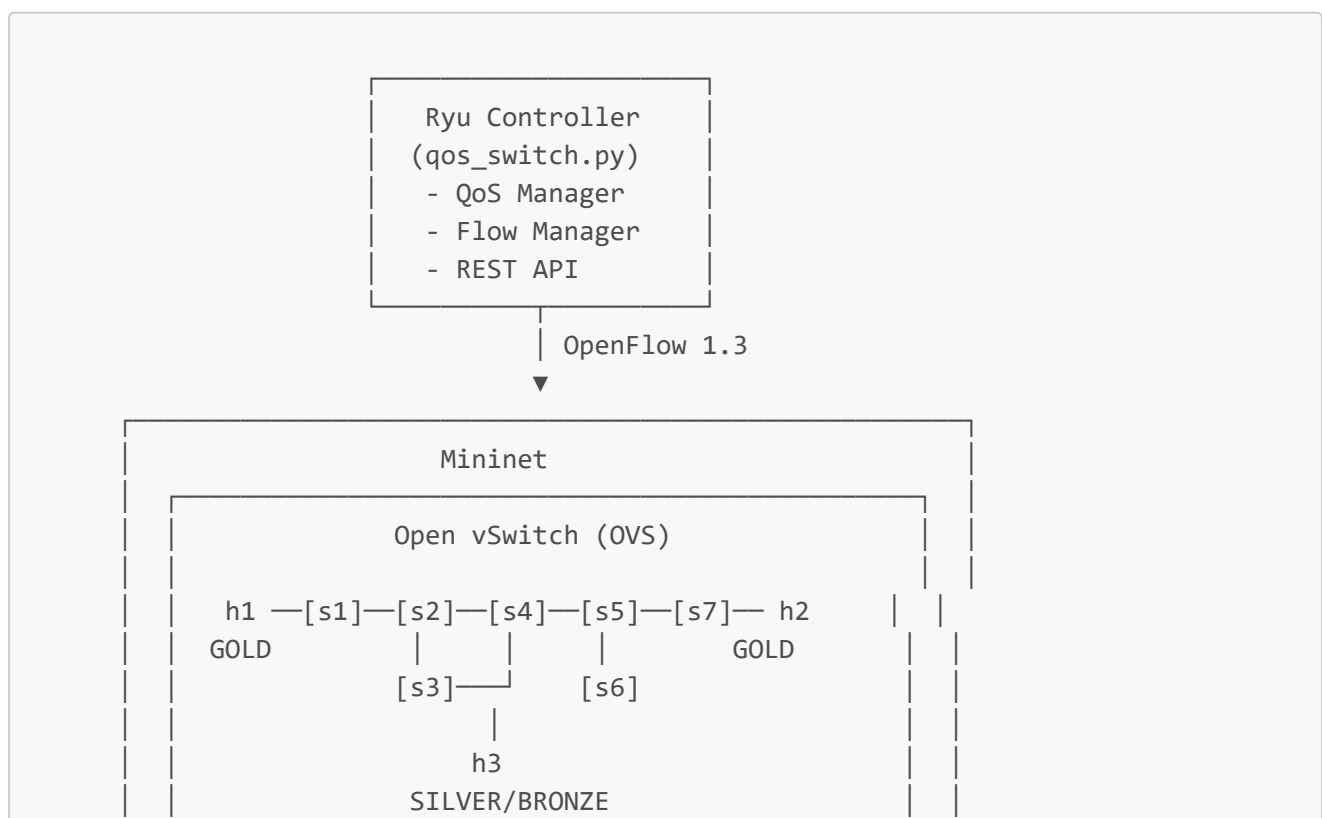
Ce projet implémente un environnement SDN (Software-Defined Networking) avec gestion de la Qualité de Service (QoS) utilisant:

- **Contrôleur Ryu** pour le plan de contrôle
- **Mininet** avec Open vSwitch pour le plan de données
- **OpenFlow 1.3** comme protocole de communication

## Objectifs

1. Déployer une topologie réseau avec 7 switches et 3 hôtes
  2. Implémenter 3 classes de service QoS (Gold, Silver, Bronze)
  3. Générer différents profils de trafic (static, bursty, oscillating)
  4. Mesurer et évaluer les performances (throughput, latency, fairness)
- 

## Architecture



## Classes de Service QoS

| Classe        | Queue | Priorité    | Bande Passante          | Flux    |
|---------------|-------|-------------|-------------------------|---------|
| <b>GOLD</b>   | 0     | Haute (0)   | 500 Mbps - 1 Gbps       | h1 ↔ h2 |
| <b>SILVER</b> | 1     | Moyenne (1) | 200 Mbps - 500 Mbps     | h1 ↔ h3 |
| <b>BRONZE</b> | 2     | Basse (2)   | Best Effort (≤200 Mbps) | Default |

## Structure du Projet

```
sdn-qos-project/
├── README.md                # Ce fichier
├── ryu_apps/
│   └── qos_switch.py        # Application Ryu avec QoS
├── scripts/
│   ├── topology.py          # Script de topologie Mininet
│   ├── traffic_generator.py  # Générateur de trafic
│   ├── metrics_collector.py  # Collecteur de métriques
│   └── run.sh                # Script d'orchestration
├── configs/
│   └── qos_config.yaml      # Configuration QoS (optionnel)
└── results/
    ├── metrics.json          # Métriques collectées
    ├── throughput_comparison.png
    ├── latency_comparison.png
    └── qos_report.txt
```

## Installation

### Prérequis

```
# Ubuntu 22.04 LTS
sudo apt update
sudo apt install -y mininet openvswitch-switch iperf3 python3-pip

# Ryu Controller
pip3 install ryu eventlet

# Visualisation (optionnel)
pip3 install matplotlib
```

## Vérification

```
# Vérifier Mininet
sudo mn --test pingall

# Vérifier Ryu
ryu-manager --version

# Vérifier OVS
ovs-vsctl --version
```

---

## Utilisation

### Méthode 1: Script automatisé

```
# Vérifier les dépendances
./scripts/run.sh --check

# Lancer la démo complète
sudo ./scripts/run.sh --full

# Arrêter et nettoyer
sudo ./scripts/run.sh --stop
```

### Méthode 2: Lancement manuel

#### Terminal 1 - Contrôleur Ryu:

```
ryu-manager --ofp-tcp-listen-port 6633 --verbose ryu_apps/qos_switch.py
```

#### Terminal 2 - Topologie Mininet:

```
sudo python3 scripts/topology.py
```

#### Terminal 3 - Tests (dans Mininet CLI):

```
# Test de connectivité
mininet> pingall

# Démarrer serveurs iperf
mininet> h2 iperf3 -s -p 5201 &
```

```
mininet> h3 iperf3 -s -p 5202 &

# Test GOLD (h1 -> h2)
mininet> h1 iperf3 -c 10.0.0.2 -p 5201 -t 30 -b 500M

# Test SILVER (h1 -> h3)
mininet> h1 iperf3 -c 10.0.0.3 -p 5202 -t 30 -b 200M
```

---

## Génération de Trafic

### Trafic Statique

```
python3 scripts/traffic_generator.py \
    --pattern static \
    --target 10.0.0.2 \
    --rate 100M \
    --duration 60
```

### Trafic en Rafales (Bursty)

```
python3 scripts/traffic_generator.py \
    --pattern bursty \
    --target 10.0.0.2 \
    --rate 100M \
    --on 5 \
    --off 2 \
    --duration 60
```

### Trafic Oscillant

```
python3 scripts/traffic_generator.py \
    --pattern oscillating \
    --target 10.0.0.2 \
    --min-rate 10M \
    --max-rate 100M \
    --period 20 \
    --duration 60
```

---

## Collecte des Métriques

### Lancer la collecte complète

```
python3 scripts/metrics_collector.py --full-test --duration 120 --output results/
```

### Générer les visualisations

```
python3 scripts/metrics_collector.py --visualize --input results/metrics.json
```

### Générer le rapport

```
python3 scripts/metrics_collector.py --report --input results/metrics.json
```

---

## Configuration QoS

### Vérifier les queues OVS

```
ovs-vsctl list queue
```

### Vérifier les règles de flux

```
ovs-ofctl -O OpenFlow13 dump-flows s4
```

### Statistiques des ports

```
ovs-ofctl -O OpenFlow13 dump-ports s4
```

---

## Métriques Mesurées

### 1. Throughput (Débit)

- Mesuré avec iperf3 en Mbps
- Comparaison entre classes Gold/Silver/Bronze

### 2. Latency (Latence)

- Mesuré avec ping (RTT en ms)
- Min/Avg/Max/Mdev par classe

### 3. Jain's Fairness Index

- Formule:  $J(x) = (\sum x_i)^2 / (n \times \sum x_i^2)$
  - Valeur entre 0 et 1 (1 = parfaitement équitable)
- 

## Dépannage

Le contrôleur ne démarre pas

```
# Vérifier les ports utilisés
sudo lsof -i :6633

# Tuer les processus existants
sudo pkill -f ryu-manager
```

Mininet ne se connecte pas au contrôleur

```
# Nettoyer Mininet
sudo mn -c

# Vérifier que le contrôleur écoute
netstat -tlnp | grep 6633
```

Les queues QoS ne fonctionnent pas

```
# Vérifier la configuration OVS
ovs-vsctl show

# Reconfigurer les queues
sudo python3 scripts/topology.py --no-cli
```

---

## Références

- [Ryu SDN Framework](#)
  - [Mininet Documentation](#)
  - [Open vSwitch Manual](#)
  - [OpenFlow 1.3 Specification](#)
- 

## Livrables

1. Code source (Ryu + scripts Python)

2. Vidéo de démonstration (2 min)
3. Slides de soutenance finale
4. Rapport de métriques

---

## Planning

| Phase         | Dates           | Statut     |
|---------------|-----------------|------------|
| Préparation   | Jusqu'au 18 Déc | ☑ Terminé  |
| Développement | 19 Déc - 5 Jan  | 📅 En cours |
| Tests         | 6 - 15 Jan      | ⌚ À venir  |
| Finalisation  | 16 - 22 Jan     | ⌚ À venir  |

### Dates clés:

- 8 Janvier: Tutorat Teams
- 22 Janvier: Soutenance finale
- 15 Février: Dépôt vidéo

---

*Projet réalisé dans le cadre du cours RSX217 au CNAM Paris*